TILUX ASM v1.0

AUTONOMOUS SYSTEM MANAGER · OFFICIAL SYSTEM REPORT · PREPARED FOR DISTRIBUTION

> *"Tilux ASM v1.0 is a voice-first autonomous desktop assistant that listens to plain spoken English, reasons about your intent, and then operates the computer itself — opening applications, running commands, handling files, and completing real work on your behalf — all without you touching the keyboard."*

1. Executive Summary

Tilux ASM v1.0 (Autonomous System Manager) is a natively developed desktop automation assistant created by Adekola Israel. It is not a chatbot that answers questions and stops there. It is an execution layer for the operating system: the user speaks a goal in ordinary language, and the system translates that goal into concrete actions on the machine, carries them out, and reports back in spoken audio.

The product is built around a single principle: the user should be able to command their computer the way they command a capable human colleague — by talking. There is no syntax to memorise, no menu to hunt through, and no need to reach for a mouse or a keyboard. The assistant is designed to run continuously in the background, surface when called, and disappear when dismissed.

The engine and automation core are proprietary to Tilux. The system is installed locally, operates against the user's own machine, and does not require a cloud dashboard to function.

2. How the Application Actually Works

At a technical level, the application runs as a continuous four-stage pipeline. Every spoken request passes through the same loop.

Stage 1 — Voice Capture

The floating Voice Awake widget holds the microphone open and streams the user's speech into a real-time speech-to-text engine. This stage is hands-free: there is no push-to-talk button and no wake-word step required once the assistant is live. The captured audio is transcribed into text as the user is still speaking.

Stage 2 — Intent Reasoning

The transcribed text is sent to the Tilux reasoning core, which interprets what the user is actually trying to achieve. This is the difference between a command parser and an assistant. If the user says "put my music on", the core understands that it must locate a media application, determine which one is installed, and issue a play command — rather than failing because no literal command named "put my music on" exists.

The reasoning core also handles ambiguity: it assesses whether the request is fully specified, whether it needs a clarifying question, or whether there is an obvious sensible default it should simply execute.

Stage 3 — Action Routing

Once an intent is settled, the system selects the fastest and most reliable execution route. It does not blindly click around the screen. It ranks its options in a strict order of preference:

- **Native command line and D-Bus interfaces** — the most reliable and fastest route, used whenever the target application exposes a programmatic interface.

- **Application shortcuts and internal search** — used when a CLI path is unavailable but the app offers a keyboard-driven entry point.
- **Simulated mouse and keyboard input** — used as the fallback for applications that expose no programmatic interface at all, replicating precisely what a human would do.

This tiered routing is what allows the same assistant to control a modern scriptable application and a stubborn legacy desktop program with equal competence.

Stage 4 — Verification and Feedback

After acting, the system does not simply assume success. It performs a lightweight perception check — reading text off the screen and confirming expected state — then reports the outcome back to the user in spoken audio. If an action fails, it is logged as a failed route so the system does not repeat a dead end in future sessions.

3. Core Capabilities

Capability Area	What It Does
Application Control	Launches, focuses, closes, and drives desktop applications. Handles both modern and legacy programs.
Web and Search	Opens browsers, runs searches, navigates to pages, and retrieves information directly.
File and Folder Management	Creates, moves, renames, organises, and inspects files and directories across the system.
Messaging	Composes and sends messages to specified

	contacts through supported desktop messaging clients.
Media Playback	Plays, pauses, skips, and controls media players on request.
Document and Report Generation	Produces formatted documents, reports, and output files on demand, then opens them for review.
System Inspection	Queries system state, running processes, storage, and configuration.
Visual Asset Creation	Generates images and produces transparent-background assets for design and web work.
Memory	Retains user preferences, personal facts, and known failed routes across sessions.

4. Personality and Communication Model

Tilux ASM is not a dry command shell. It speaks. Every response is delivered as real-time spoken audio through the Voice Awake widget, in a configurable tone of voice. The user selects the register that suits them — Professional, Sassy Gen-Z, Joking, Supportive, Philosophical, Poetic, Robotic, Chaotic, and others — and the assistant holds that register consistently.

The assistant understands both standard English and Nigerian Pidgin, and always replies in clear English unless told otherwise. It is deliberately tuned for conversational efficiency: replies are short, direct, and free of visual formatting such as bullet lists or code blocks, because those do not translate into speech.

It also handles the realities of live voice input. When a microphone clip is garbled or truncated, the assistant says so honestly rather than guessing at a wrong answer — a small design decision with a large effect on trust.

5. Task Handling Model

Requests are handled in three distinct modes, chosen automatically based on complexity.

Mode 1 — Immediate Execution

Simple and multi-step but straightforward tasks are executed immediately, without asking the user to confirm or clarify. The system does not stall on unnecessary questions.

Mode 2 — Conversational Response

Questions and general conversation are answered directly and concisely, in voice. The assistant distinguishes between a request to build something and a request to simply discuss something.

Mode 3 — Deep Workspace Reasoning

Genuinely heavy engineering work — building a full application, large-scale code refactoring, extensive code generation — is escalated out of voice mode and routed into the desktop workspace chatbox, which handles long-form reasoning and multi-step software development. This keeps voice interaction fast while keeping ambitious work possible.

6. Architecture at a Glance

Layer	Responsibility
Voice Awake Widget	Floating always-available interface for microphone capture and spoken audio playback.
Reasoning Core	Proprietary Tilux engine that interprets intent and plans the action sequence.
Action Router	Selects and executes the optimal route: CLI / D-Bus, then shortcut, then mouse and keyboard.
Perception Layer	Reads screen state via OCR to verify that actions succeeded before reporting back.
Memory Store	Persists user preferences, facts, and failed-route logs between sessions.
Workspace Chatbox	Deep reasoning environment for software development and long-form engineering tasks.
Settings Layer	Live configuration of tone, voice, speech rate, and administrative credentials.

7. Design Principles

- **Execution over conversation.** The measure of the assistant is work completed, not words produced.
- **No unnecessary friction.** If a request can be done, it is done — not queued behind questions.
- **Honesty about failure.** Uncertain inputs are surfaced, not papered over with confident guesses.
- **Route efficiency.** The most reliable path is always preferred over the most obvious one.
- **Local first.** The system operates on the user's own machine and under the user's own control.
- **Graceful dismissal.** The assistant appears when needed and closes instantly on request.

8. Ownership and Attribution

Tilux ASM v1.0 is developed natively by **Adekola Israel** (@Tilux). The engine and automation core are proprietary to Tilux. The product is designed, built, and maintained as a first-party desktop automation platform rather than an assembly of third-party assistant frameworks.

9. One-Line Descriptions (For Use)

Shortest: Tilux ASM v1.0 is a voice-driven autonomous assistant that operates your computer for you.

Product form: Tilux ASM v1.0 listens to plain spoken English, decides what needs to happen, and then performs it directly on your machine — opening apps, running commands, managing files, and reporting back out loud.

Technical form: Tilux ASM v1.0 is a proprietary desktop automation engine that combines real-time speech recognition, intent reasoning, tiered action routing

across CLI and GUI layers, and screen verification into a single hands-free control loop.

Report prepared by Tilux ASM v1.0 · Developed natively by Adekola Israel (@Tilux) · Engine and automation core proprietary to Tilux.